

RTL Design Sherpa

CDC Counter Display on the Nexys A7 The FPGA System: Harness, Builds, Flow -- Version 1.0

September 2026

Table of Contents

1 Document Information....8
1.1 Related documents....8
1.2 Terminology....9
1.3 Revision History....9
1.4 How to read this book....9
2 The System....11
2.1 What this project is for....11
2.1.1 Figure 1.1: The bench....11
2.2 The board....11
2.3 The asymmetry between the two builds....12
2.4 Why a demonstration needs the broken version....13
3 The Harness....14
3.1 The shape of it....14
3.1.1 Figure 2.1: The harness block map....14
3.2 The front end is borrowed, deliberately....14
3.3 The global registers....15
3.4 The per-counter block, times four....16
3.5 The clock tree....17
3.6 The five crossings....17
4 Two Builds, Two Designs....19
4.1 The short answer, and it is the opposite of the pumice system's....19
4.1.1 Figure 3.1: Two builds, two designs....19

4.2 What each build actually compiles....20
4.3 Why that is the right structure here....20
4.4 The dispatcher, and where the flow logic lives....21
4.5 One compile closure per build, and it is the filelist....21
5 Host Tools and Flow....23
5.1 The tools....23
5.2 The watch-fail flow....24
5.2.1 Figure 4.1: Breaking a CDC on purpose, and watching the display say so 24
5.3 Step 0: prove the link before anything else....24
5.4 Step 1: the control group is not optional....25
5.5 Step 2: break exactly one thing....25
5.6 Step 3: the sweep is what makes the failure appear....25
5.7 Steps 4 and 5: two views of the same failure....25
5.8 Reproducibility....26

List of Figures

Figure 1.1: The bench.....11
Figure 2.1: The harness block map.....14
Figure 3.1: Two builds, two designs.....19
Figure 4.1: Breaking a CDC on purpose, and watching the display say so.....24

List of Tables

Table 1: Table 0.1: Document information....8
Table 2: Table 0.2: Related documents....8
Table 3: Table 0.3: Revision history....9
Table 4: Table 1.1: The two builds are two designs....12
Table 5: Table 2.1: The global registers....15
Table 6: Table 2.2: One counter's registers (four identical blocks)....16
Table 7: Table 2.3: The five clock sources per counter....17
Table 8: Table 2.4: The five value-out crossings....17
Table 9: Table 3.1: The two compile closures....20
Table 10: Table 4.1: The host tools....23

List of Waveforms

No waveforms in this document.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

1 Document Information

Table 0.1: Document information

Field	Value
Title	CDC Counter Display on the Nexys A7 – The FPGA System
Version	1.0
Date	2026-09-29
Scope	The CDC demonstration system: the board, the two builds and how far apart they are, the harness around the counters, the five CDC modes, and the watch-it-fail flow
Not in scope	The CDC primitives themselves (`rtl/cdc/`, documented per module under `docs/markdown/`); the operator procedure (`docs/cdc_demo_guide/`, `docs/HARNESS.md`)
Status	Current

1.1 Related documents

Table 0.2: Related documents

Document	Where	What it gives you
CDC demo operator guide	`docs/cdc_demo_guide/`	the chaptered guide for someone at the board
Harness procedure	`docs/HARNESS.md`	CSR map plus the scripted demo procedure
CDC failure taxonomy	`docs/CDC_DEMO_TODO.md`	the modes of CDC failure, and “why it sometimes works anyway”
Project README	`README.md`	the two-build table and the build/program commands
pumice system book	`../../pumice/docs/`	the same book for the

Document	Where	What it gives you
	`pumice_fpga_system/`	other Nexys A7 project, where the harness is shared rather than per-build

1.2 Terminology

CDC Clock domain crossing. Moving a signal from one clock to another where the two have no fixed phase relationship.

Harness Everything in the bitstream that exists to drive and observe the thing being demonstrated. `build-demo` has one; `build-phase1` deliberately does not.

Counter domain `cdc_counter_domain.sv` – one counter on its own clock, with a selectable value-out crossing. Instantiated four times.

Mode 0 / NO_CDC The value-out path built wrong on purpose: a raw flop per bit, no Gray coding, no `ASYNC_REG`. The subject of the headline demonstration.

Control group The counters left in a correct mode during an experiment, so that a visible failure cannot be blamed on the board.

1.3 Revision History

Table 0.3: Revision history

Version	Date	Change
1.0	2026-09-29	First edition.

1.4 How to read this book

Chapter 1 is the bench, and its one surprise: only one of the two builds needs the host at all. Chapter 2 is the harness – the CSR map, the four counter domains, the clock tree, and the five crossings you can select per counter. Chapter 3 is the comparison this project exists to support, and it answers the harness-RTL question in the opposite direction from the pumice system on the same board. Chapter 4 is the watch-it-fail flow and why each step of it is shaped the way it is.

Every diagram is a Graphviz source under `assets/graphviz/`, rendered to PNG by `regenerate_all_graphviz.sh`.

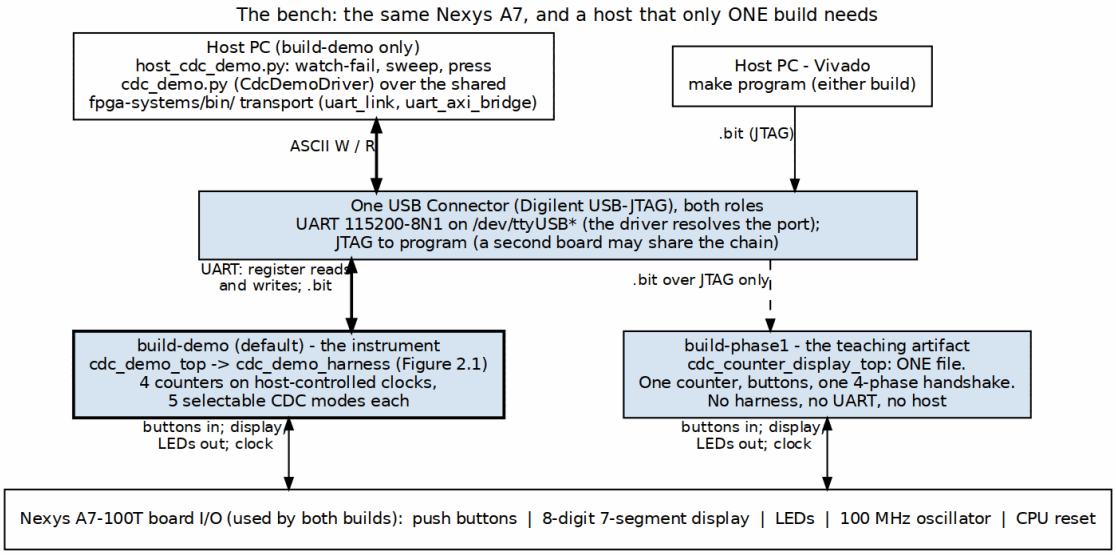
2 The System

2.1 What this project is for

This is a teaching instrument. It demonstrates clock domain crossing done correctly and done incorrectly, on real hardware, with the incorrect version failing visibly on demand. The thing being demonstrated is a counter whose value has to cross from its own clock into the clock that drives the display.

That framing matters for the rest of the book, because it makes the *display* the measuring instrument. There is no CRC and no performance counter here. The experimental result is what the 7-segment digits show.

2.1.1 Figure 1.1: The bench



The bench: the same Nexys A7, and a host that only one build needs

Source: 01_board_and_host.dot

2.2 The board

The same Nexys A7-100T as the pumice DDR2 system: Artix-7 XC7A100T-1CSG324C, one 100 MHz oscillator, one USB connector carrying both the JTAG chain and the UART. What this project uses that the DDR2 system does not is the human interface – five push buttons and the 7-segment display – and what it does not use at all is the DDR2 part.

Port discovery is the same problem and has the same answer: the `ttyUSB` numbering drifts and a second board may share the chain, so the host driver resolves the port rather than trusting a constant. The driver carries that comment where the logic lives.

2.3 The asymmetry between the two builds

The one structural fact to take from this chapter is that the two builds are not two configurations of one design. They are two designs:

Table 1.1: The two builds are two designs

	`build-phase1`	`build-demo` (default)
Top	`cdc_counter_display_top`	`cdc_demo_top`
RTL files in the build	one	three, plus a generated CSR block
Counters	one	four, each on its own clock
CDC crossings available	one, fixed	five, selectable per counter
Host	none – nothing to drive	required
Stimulus	a human pressing BTNC	buttons *or* injected register writes
What it shows	a correct crossing working	a correct crossing and a broken one, side by side

`build-phase1` is the original: a debounced button in a 10 Hz domain, a counter, and a 4-phase handshake carrying the value into a 1 kHz display domain. You walk up to the board, press a button, and the number goes up. It needs no host because there is nothing to configure.

`build-demo` is the instrument. It has four counters so that three of them can be a control group; it has host-settable clocks so a crossing can be pushed until it breaks; and it has five selectable crossings so the broken one can be compared against four correct ones without rebuilding.

Note: the project Makefile is a dispatcher, not a build. Targets go to `build-demo` by default and `BUILD=phase1` selects the other. All flow logic lives in the repo-wide `make/fpga_flow.mk`; the per-build Makefiles set variables and nothing else. That is why adding a third build would not mean copying a build system.

2.4 Why a demonstration needs the broken version

A correct CDC is invisible. It works, and a reader learns nothing from watching it work, because they cannot tell a correct crossing from a lucky one.

The failure is the content. But an unsafe crossing is not reliably broken – it is *probabilistically* broken, and at low clock rates it mostly works. A demo that shows mode 0 at 6 MHz demonstrates

that the wrong design is fine, which is worse than showing nothing. Making the failure appear on demand is the actual engineering problem this system solves, and Chapter 4 is how.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

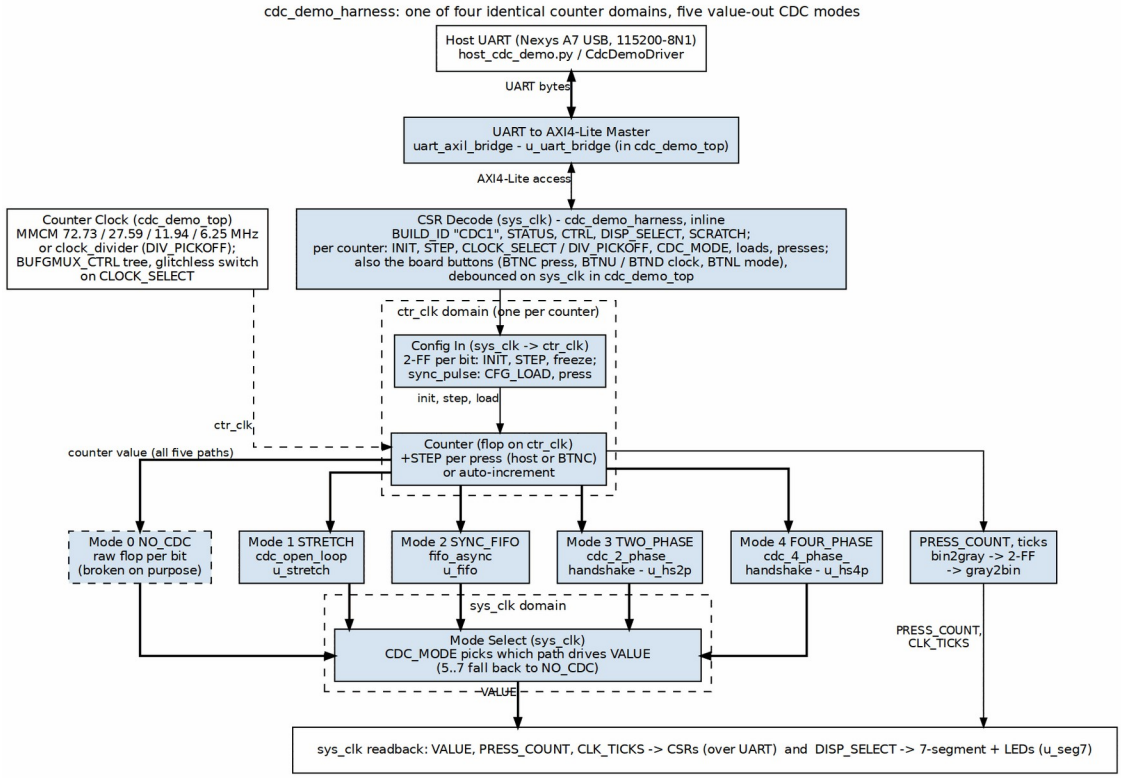
---

3 The Harness

3.1 The shape of it

`cdc_demo_harness` is a UART bridge, a generated CSR block, four identical counter domains on four different clocks, and a display multiplexer. The host writes configuration, the counters count, and the crossings under test carry their values into the system clock where the host and the display can see them.

3.1.1 Figure 2.1: The harness block map



cdc_demo_harness: four identical counter domains, five CDC modes each

Source: 02_harness_blocks.dot

3.2 The front end is borrowed, deliberately

`uart_axil_bridge` is the same component the pumice DDR2 harness uses, and the host-side transport in `projects/fpga-systems/bin/` is the same too. Nothing about the serial link is specific to CDC.

That reuse is the point: a project whose subject is clock domain crossing should not also be inventing a serial protocol. It also means the port-discovery and framing bugs are fixed once, for every board on the bench.

3.3 The global registers

`cdc_demo_csr.rdl` generates the CSR block. The global half is small:

Table 2.1: The global registers

Register	Fields	Purpose
`BUILD_ID`	`0x43434331`	ASCII `CDC1`. Proves which bitstream you are talking to
`STATUS`	`alive0..3`, `uart_rx`, `uart_tx`, `any_written`, `reset_ok`	liveness per counter, link activity, and whether anything has been configured since reset
`CTRL`	`soft_reset`, `freeze`, `ignore_btn`	pulse a reset, stop all counters, or lock out the physical buttons
`DISPLAY`	`sel[1:0]`	which counter's value drives the 7-segment display
`SCRATCH`	32 bits	read-write, host link sanity

Three of those exist because of specific failure modes at the bench:

- **`BUILD_ID`** – the cheapest wasted afternoon is measuring a stale bitstream. If `CDC1` does not read back, stop.
- **`any_written`** – distinguishes "the demo is showing defaults" from "the demo is showing what I configured". Without it, a host script that silently failed to write looks identical to one that worked.
- **`ignore_btn`** – a physical button press during a scripted sweep is an uncontrolled input. `ignore_btn` makes the experiment reproducible by taking the human out of it.

`freeze` is the other quiet one: it stops every counter at once so a displayed value can be read carefully rather than glimpsed.

3.4 The per-counter block, times four

Each of the four counters has an identical register block:

Table 2.2: One counter's registers (four identical blocks)

Register	Fields	Purpose
`CLK`	`clock_select[2:0]`, `div_pickoff[12:8]`	which clock this counter runs on; `0..3` select an MMCM output, `4` selects the divided clock, and `div_pickoff` then chooses the divisor tap
`INIT`	`value[15:0]`	the value `CFG_LOAD` will load
`STEP`	`value[15:0]`	how much a press advances the counter
`CFG_LOAD`	strobe	pulse: load `INIT`
`HOST_PRESS`	strobe	pulse: inject one virtual button press
`VALUE`	`value[15:0]`	the counter's current value, **crossed into `sys_clk` by the mode under test**
`PRESS_COUNT`	`value[15:0]`	debounced press count, crossed **Gray-coded**

Two design decisions are worth drawing out.

`STEP` is not always 1. A counter advancing by 1 changes one or two bits per press, and a multi-bit crossing hazard needs many bits changing at once to show itself. A `STEP` that flips several bits at a time makes the failure far more likely per event, which is the difference between a demo that fails in seconds and one that fails in an hour.

`HOST_PRESS` exists so the stimulus is not a finger. A scripted press is repeatable, can be issued thousands of times, and can be issued faster than a human can press. The physical buttons remain for the walk-up demonstration.

Important: `VALUE` and `PRESS_COUNT` cross differently on purpose. `VALUE` crosses by whatever mode is selected – including the broken one – while `PRESS_COUNT` always crosses Gray-coded.

During a failure the press count stays trustworthy while the value does not, and that contrast *within one counter* is what shows the reader the bug is in the crossing and not in the counting.

3.5 The clock tree

The clocks are the experiment's independent variable, so they are generated deliberately rather than divided down casually:

Table 2.3: The five clock sources per counter

Index	Source	Frequency
0	MMCM `CLKOUT0`, divide 11	72.73 MHz (fastest)
1	MMCM `CLKOUT1`, divide 29	27.59 MHz
2	MMCM `CLKOUT2`, divide 67	11.9 MHz
3	MMCM `CLKOUT3`, divide 128	6.25 MHz (slowest MMCM output)
4	`clock_divider`, `DIV_PICKOFF` 0..31	host-selectable, the fine-grained sweep

The MMCM runs at a VCO of 800 MHz from the 100 MHz input with a feedback multiplier of 8.0 and `DIVCLK_DIVIDE` of 1. The four outputs are chosen to be mutually unrelated rather than neat multiples: a crossing between clocks with a simple integer relationship can pass by luck, and luck is exactly what this project is trying to remove.

Important: per counter, a tree of four `BUFGMUX_CTRL` instances selects the source, and that switching is *glitchless* by the primitive's guarantee. This is what allows the host to change a counter's clock while it is running, which the sweep in Chapter 4 depends on. A naive clock mux would produce a runt pulse on every change and the resulting corruption would be indistinguishable from the CDC failure being demonstrated – the apparatus would be generating the result.

3.6 The five crossings

The value-out path is the thing under test, and it is selectable:

Table 2.4: The five value-out crossings

Mode	Name	Implementation	Why it is here
0	`NO_CDC`	a raw flop per bit;	**broken on**

Mode	Name	Implementation	Why it is here
		no Gray, no `ASYNC_REG`	**purpose** – the demonstration
1	`STRETCH`	`cdc_open_loop`: source holds data and valid long enough for the destination to sample	the cheapest correct answer when the source can be made to wait
2	`SYNC_FIFO`	`fifo_async`, `FIFO_DEPTH = 16`	the general answer: every event survives, in order
3	`TWO_PHASE`	`cdc_2_phase_handshake`, toggle / NRZ	correct, and cheaper per transfer than 4-phase
4	`FOUR_PHASE`	`cdc_4_phase_handshake`, classic return-to-zero	the conservative textbook crossing; the one `build-phase1` uses

Modes 3 and 4 snapshot the counter every `HANDSHAKE_INTERVAL = 256` counter-clock cycles. A handshake has a round-trip cost, so it cannot forward every increment; it forwards a consistent *sample*. That is the right trade for a displayed value and the wrong one for an event count – which is why the press count does not use it.

Note: modes 1 through 4 are all correct, and they are not interchangeable. Their differences are throughput, latency and whether every event or only the latest value survives. Selecting between four correct answers and seeing that they all display the same number is a quieter lesson than mode 0, and it is available in the same bitstream.

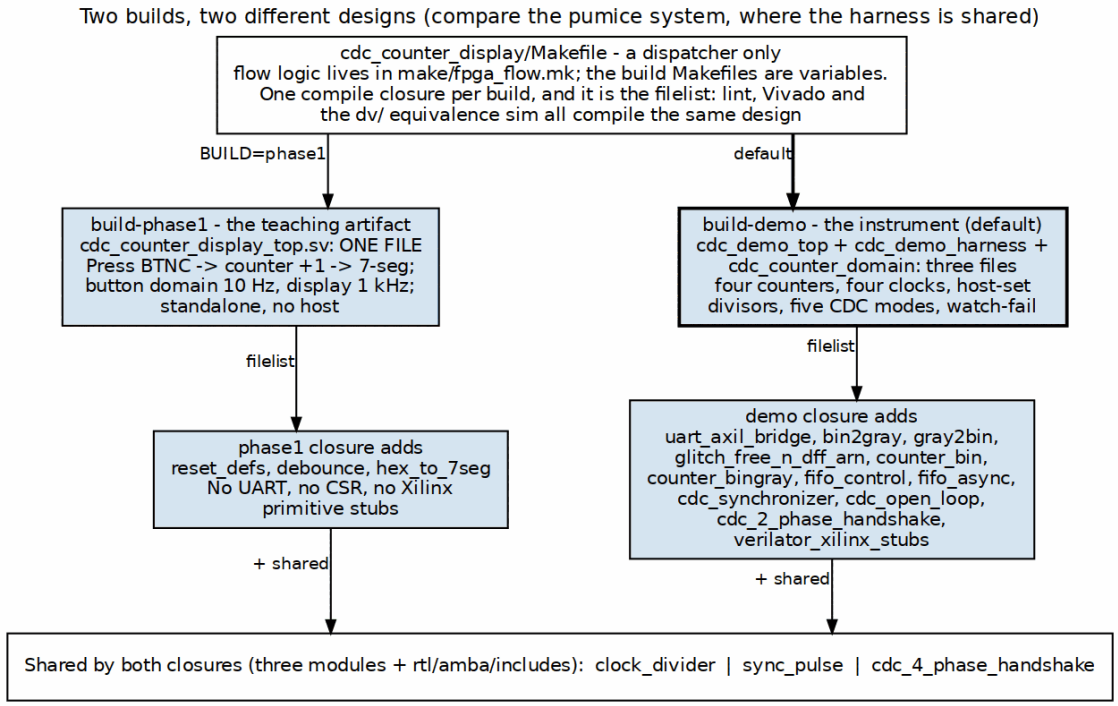
4 Two Builds, Two Designs

4.1 The short answer, and it is the opposite of the pumice system's

Yes – the harness RTL is different per build here, completely. `build-phase1` has no harness at all. It is one RTL file with no UART, no CSR block and no host interface. `build-demo` is three files plus a generated CSR block, and the dependency closures of the two builds overlap in three modules.

This is worth stating plainly because the other Nexys A7 project answers the same question the other way. The pumice DDR2 system has *one* harness source that every one of its five targets compiles; only the DUT, the PHY or the debug cores change. Both answers are right for their project, and the reason they differ is the reason this chapter exists.

4.1.1 Figure 3.1: Two builds, two designs



Two builds, two different designs

Source: 03_build_variants.dot

4.2 What each build actually compiles

Table 3.1: The two compile closures

	build-phase1	build-demo
Build RTL	cdc_counter_display_top.sv	cdc_demo_top.sv, cdc_demo_harness.sv, cdc_counter_domain.sv
Generated	none	cdc_demo_csr from cdc_demo_csr.rdl
Repo closure	reset_defs, clock_divider, debounce, hex_to_7seg, sync_pulse, cdc_4_phase_handshake	uart_axil_bridge, bin2gray, gray2bin, sync_pulse, glitch_free_n_dff_arn, counter_bin, counter_bingray, fifo_control, fifo_async, cdc_synchronizer, cdc_open_loop, cdc_2_phase_handshake, cdc_4_phase_handshake, clock_divider
Xilinx primitive stubs	none needed – the top instantiates none	IBUF, BUFG, BUFGMUX_CTRL, MMCME2_BASE

The shared part is three modules – `clock_divider`, `sync_pulse` and `cdc_4_phase_handshake` – plus the AMBA include directory. Everything else is disjoint. These are two designs that share a project directory, a board and a build flow, and nothing else.

4.3 Why that is the right structure here

The pumice system shares its harness because its builds are *the same experiment on different subjects*: swap pumice for LiteDRAM and the numbers must stay comparable, so the apparatus must not move. Sharing is what makes the comparison valid.

Here the two builds are *different experiments*:

- `build-phase1` answers “can a beginner see a correct CDC work on real hardware, with nothing but a board and a USB cable for power?” Adding a

UART harness to it would make it worse. The absence of a host is the feature: there is no configuration to get wrong and nothing between the button and the display.

- `build-demo` answers "can we make an unsafe CDC fail on demand, visibly, with a control group?" That needs four counters, host-settable clocks and selectable crossings, none of which phase 1 can express.

Forcing one harness onto both would compromise both: phase 1 would stop being a standalone artifact, and the demo would inherit constraints from a teaching example. Keeping them separate costs one extra top-level file and a dispatcher.

Note: this is why `build-phase1` is kept rather than deleted as superseded. It is not an earlier version of `build-demo`; it is the simplest thing that demonstrates the concept, and that has its own value.

4.4 The dispatcher, and where the flow logic lives

`cdc_counter_display/Makefile` selects a build and does nothing else. Targets go to `build-demo` by default; `BUILD=phase1` selects the other:

```
make bitstream                  # build-demo
make program                    # build-demo
make bitstream BUILD=phase1     # build-phase1
make program   BUILD=phase1
```

All flow logic – synthesis, programming, lint, cleaning – lives in the repo-wide `make/fpga_flow.mk`. The per-build Makefiles set variables only.

That split is what keeps two builds from becoming two build systems. The same `fpga_flow.mk` drives the pumice builds on this board and the STREAM and RAPIDS builds on the Genesys 2, so a fix to the flow reaches every project at once.

4.5 One compile closure per build, and it is the filelist

Each build has exactly one filelist, and three different tools consume it:

- `make lint` flattens it;
- Vivado's `create_project.tcl` expands it, dropping the verilator-only primitive stubs so synthesis sees the real unisims;
- the `dv/` equivalence simulation `-f` includes it.

Important: that is deliberate and it is the whole defence against the classic FPGA failure of linting one design, simulating a second and synthesizing a third. Because all three read the same list, they cannot disagree about what the design is. Both build filelists carry that statement in their header

comment, which is the right place for it – the next person tempted to add a source directly to the Vivado project reads why not.

The stub handling is the one subtlety. `verilator_xilinx_stubs` provides `IBUF`, `BUFG`, `BUFGMUX_CTRL` and `MMCME2_BASE` so that lint and simulation can elaborate the top. They are guarded by `ifdef VERILATOR` *and* dropped by the Vivado tcl – belt and braces, because a stub reaching synthesis would silently replace a real clock primitive.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Host Tools and Flow

5.1 The tools

Table 4.1: The host tools

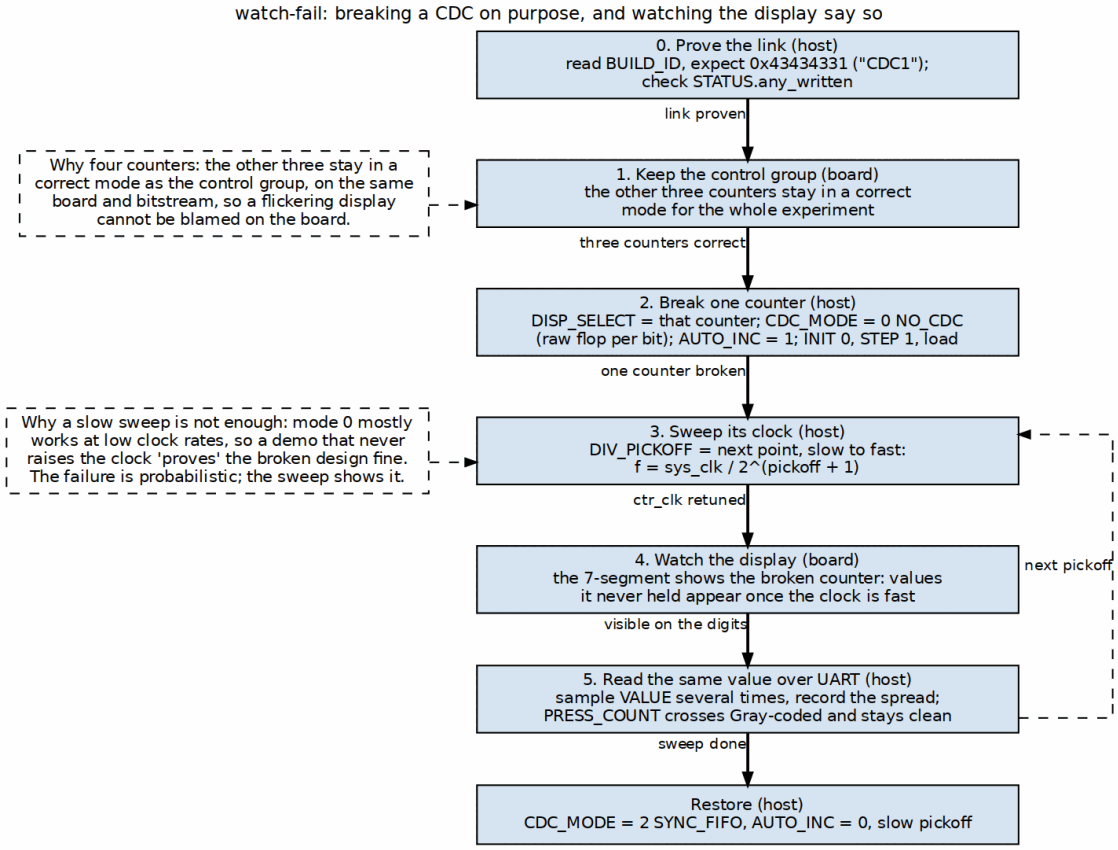
Program	Role
`host_cdc_demo.py`	the CLI: `watch-fail`, sweeps, presses, per-counter configuration
`cdc_demo.py`	`CdcDemoDriver` – the register-level API the CLI drives
`fpga-systems/bin/`	`uart_link`, `uart_axi_bridge`, `board` – the shared transport, identical to the pumice system's

The headline invocation is one line:

```
make host-cdc_demo ARGS="watch-fail --counter 2"
```

5.2 The watch-fail flow

5.2.1 Figure 4.1: Breaking a CDC on purpose, and watching the display say so



watch-fail: breaking a CDC on purpose, and watching the display say so

Source: 04_measurement_path.dot

The sequence is: prove the link, leave three counters correct, break one, sweep its clock, watch the display, and read the same value back over UART. Each step is there for a reason, and the reasons are the content of this chapter.

5.3 Step 0: prove the link before anything else

Read `BUILD_ID` and expect `0x43434331`. A host that cannot read `CDC1` back is talking to a different bitstream, and every subsequent observation is fiction. Then check `any_written` in `STATUS`: it distinguishes a demo showing configured values from one showing power-on defaults, which look identical on the display.

5.4 Step 1: the control group is not optional

Three counters stay in a correct mode for the whole experiment. This is the step most likely to be skipped and the one that makes the result mean anything.

Without it, “the display is flickering” has many explanations: a marginal board, a bad USB cable, a power supply, a timing failure somewhere else entirely, or the CDC bug being demonstrated. With three correct counters running in the same bitstream, on the same die, at the same temperature, at the same instant, all of those explanations are eliminated at once. Three clean digit groups beside one scrambled group is evidence; one scrambled group alone is an anecdote.

5.5 Step 2: break exactly one thing

Set the chosen counter’s value-out path to mode 0, `NO_CDC`: a raw flop per bit, no Gray coding, no `ASYNC_REG` attribute. Nothing else changes.

It is worth being precise about what is wrong with it. The flops will mostly settle; the problem is that the counter’s bits do not all change at the same instant, and a destination clock that samples during the transition captures a mixture of old and new bits. The result is a value the counter never held. With no Gray coding, a single increment can change many bits, so the window for a corrupt sample is wide.

5.6 Step 3: the sweep is what makes the failure appear

This is the step that distinguishes a working demonstration from a misleading one.

Important: mode 0 is a real design that mostly works. At 6.25 MHz against a system clock, the raw flops usually have time to settle and the value reads back correctly. A demonstration that sets mode 0 and stops has just shown the audience that the unsafe design is fine – the most damaging possible outcome for a teaching tool.

So the host sweeps that counter’s clock from slow to fast through `clock_select` and `div_pickoff`, while the other three hold. The failure is probabilistic in the ratio of the two clock rates and in how many bits changed, so the sweep walks the probability up until the failure is continuous and obvious. This is also why the clock mux has to be glitchless: if changing the clock itself produced a runt pulse, the sweep would generate corruption indistinguishable from the corruption being studied.

5.7 Steps 4 and 5: two views of the same failure

The display shows the broken counter’s value – `DISPLAY.sel` points the 7-segment at it – and at the fast end of the sweep the digits show values the counter never held.

Reading the same counter over UART adds the part that identifies the bug rather than just exhibiting it. `VALUE` crosses by the selected (broken) mode; `PRESS_COUNT` always crosses Gray-coded. During the failure the press count remains correct while the value does not. The counter is

therefore counting correctly and the crossing is losing information – which is the conclusion the demonstration exists to support, and it is visible in two registers of the same block.

`CTRL.freeze` is useful here: stopping every counter lets a corrupt value be read carefully instead of glimpsed, and confirms that the corrupt value was *sampled* rather than merely displayed too briefly to read.

5.8 Reproducibility

Two features exist only to make a scripted run repeatable:

- **`ignore_btn`** locks out the physical buttons, so a bystander pressing BTNC cannot perturb a sweep.
- **`HOST_PRESS`** replaces the finger: presses become register writes, issued as fast and as often as needed, identically every run.

Note: the same discipline that applies to the DDR2 system applies here. A simulation run against a stale build directory reports success for the previous design; `make clean-all` before a run whose result you intend to believe. The `dv/` equivalence simulation shares the build's filelist for exactly this reason – so what you simulated is what you will program.